TotallyRealEstate Phase 2 Demo Guide

A 5 minute 15 second demonstration with English instructions and narration

The main narration is 463 words. The schedule includes time for clicks and window changes. Practise at 110-120 words per minute. The optional transaction adds about 30 seconds, for a total of 5 minutes 45 seconds.

All project paths start inside the extracted Phase2_Implementation folder. The SQL files use labelled sections so you can select the required query directly.

Recording timeline

Time	Screen action	Evidence
00:00-00:20	Open README and introduce the scope	Three data representations
00:20-00:55	Show tables and run D0 in Business_Queries.sql	11 tables and 2,408 rows
00:55-01:20	Run Q01 and explain the view results	75 marketed listings
01:20-01:50	Run Q08 and show cumulative sales value	10 monthly rows
01:50-02:20	Run Q12 and explain the correlated subquery	23 below-average listings
02:20-03:50	Update and restore a note, then run F01	One update and 3 notes
03:50-04:55	Show the object and JSON, then run J01 and J02	Nested reads and filtering
04:55-05:15	Return to README and explain data ownership	Authority and tradeoffs

Before recording

1. Use the original sample database. If setup is incomplete, run SQL_Scripts files 01 through 05, then 07_views.sql and 08_stored_procedures.sql. Next run OO_JSONB_Mapping/00_schema.sql and 01_load_snapshots.sql before recording.

2. In pgAdmin, open Business_Queries.sql, Transaction_Examples.sql and JSONB_Queries.sql. Run D0 in Business_Queries.sql: expect 2,408 business rows. Check JSONB with SELECT count(*) FROM tre_oo.listing_snapshot; expect 250. Load the sample data only once.

3. Open two Firestore browser tabs: the note-0001 document and the F01 query configured on page 3. During recording, click Run query to show actual execution.

4. In an editor, open models.py and listing_snapshots.json from OO_JSONB_Mapping. Switch between prepared tabs and use a readable font size.

5. Read only the paragraphs labelled Narration. Rehearse once with a timer. If the recording exceeds six minutes, omit the optional transaction and the second Firestore query.

Expected results use the original sample data. If your results differ, check the database connection and data before reading the figures aloud.

PostgreSQL walkthrough

Introduction

00:00-00:20 Open README.md and show the project structure or Data ownership table.

Narration Hello. This demonstration presents the Phase Two implementation for TotallyRealEstate. I will show the PostgreSQL schema and business queries, the Cloud Firestore notes, and a Listing object stored as JSONB.

Schema and data

00:20-00:55 In pgAdmin, expand Schemas > tre > Tables to show the 11 tables. Open SQL_Scripts/01_schema.sql and point to a primary key and a foreign key. Run the complete D0 section in Query_Portfolio/Business_Queries.sql to display the row counts and total.

Narration The relational database contains eleven business tables and two thousand four hundred and eight synthetic rows, including eight internal staff members. Here are the tables and their row counts. Primary keys, foreign keys and checks protect data integrity. Separate indexes and triggers enforce rules such as one accepted offer per listing and showing capacity.

Q01 Active listings

`Query_Portfolio/Business_Queries.sql  section Q01`

00:55-01:20 Find BEGIN Q01 and execute its complete SELECT. Expect 75 rows. Point to listing_agent, seller_count and seller_names. The installed view definition is in SQL_Scripts/07_views.sql.

Narration This first query shows the listings that are currently active or under offer. It combines property details, the listing agent and the sellers through a reusable view. Seller information is aggregated first, so a listing with two sellers still appears only once. The fixture returns seventy-five listings.

Q08 Monthly sales

`Query_Portfolio/Business_Queries.sql  section Q08`

01:20-01:50 Find BEGIN Q08 and execute its complete WITH...SELECT. Expect 10 rows. Point to LAG and the running SUM. The final cumulative value is CAD 54,205,030.00. September and October remain visible with zero sales.

Narration The next query summarizes completed sales by month. It uses grouping, a calendar sequence and window functions to calculate the previous month and a running total. It also keeps months with no sales. The final cumulative value is about fifty-four point two million Canadian dollars. This is synthetic property transaction value, not company revenue.

Q12 Internal price comparison

`Query_Portfolio/Business_Queries.sql  section Q12`

01:50-02:20 Find BEGIN Q12 and execute its complete WITH...SELECT. Expect 23 rows. The first is listing 102: asking price CAD 456,500.00 and internal segment average CAD 646,250.00. Point to the inner SELECT AVG and explain the city and property-type comparison.

Narration This query finds active listings priced below the average for the same city and property type. A correlated subquery calculates that internal benchmark. Twenty-three listings match. For example, listing one hundred and two is priced below the Calgary condo average. These results compare our sample inventory, rather than estimate real market values.

Firestore walkthrough

02:20-03:50 Sign in to Firebase Console with access to this project. Demonstrate an UPDATE and execute a query. The full CRUD source is in NoSQL_Prototype/firestore_demo.mjs.

```
Project: totallyrealestate-phase-2
Collection: tre_phase2_showing_notes
Document: tre-phase2-note-0001
```

Panel view

1. Open Cloud Firestore > Data > Panel view. Select the collection and document listed above. Expand feedback, follow_up and tags, and point to the maps, array and timestamp fields.

2. Find status inside follow_up. The original value is PENDING; check it before recording. Click its edit button, change the string to DONE, and save. Show the new value.

3. Edit the same field again, restore PENDING, and save. Perform these updates during recording. Keep showing_id unchanged and retain the document.

Query builder F01

4. Switch to the prepared second tab: Data > Query builder. If needed, configure it below. Use Add to query for Where and Order by.

Setting	Value
Query scope	Collection
Path	tre_phase2_showing_notes
Where	Property showing_id Operator ==
Value type and value	number 1 (use a numeric value)
Order by	Property created_at descending
Limit	100 (keep the default)

5. Click Run query. Expect three documents with ID endings 0003, 0002 and 0001, ordered newest first. Use a numeric value for showing_id.

The corresponding SDK query is in the QUERIES F01 F02 F03 section of NoSQL_Prototype/firestore_demo.mjs. Run the visual query in Firebase Console; the source is available for reference.

Narration

Narration Firestore stores showing notes because the text, tags and feedback can vary between visits. The collection contains sixty documents. This note links to a PostgreSQL showing and includes nested feedback and follow-up maps, arrays, and timestamp fields. I will change the follow-up status from pending to done, save it, and then restore the original value. This demonstrates an update without leaving the fixture changed. Next, I filter for showing ID one and sort by creation time, newest first. The query returns three notes in reverse time order. PostgreSQL remains the source for operational data, while Firestore holds the note content.

Optional second query

If time allows, run F02: replace the Where filter with follow_up.required == true, using type boolean. Keep created_at as Order by and select ascending. Expect 20 documents. F01 alone is sufficient for the main route.

Console: https://console.firebase.google.com/project/totallyrealestate-phase-2/firestore/databases/-default-/data?hl=en

Object mapping and closing

03:50-04:55 Switch back to the editor and pgAdmin. Follow this sequence.

`OO_JSONB_Mapping/models.py`

1. Find class Listing. Show property, agent, sellers, offers, and the to_dict/from_dict methods. Explain the composition relationships.

`OO_JSONB_Mapping/listing_snapshots.json`

2. Show the first object, listing 1, with nested property, agent, sellers and offers. Continue with PostgreSQL queries to demonstrate the stored JSONB.

`OO_JSONB_Mapping/JSONB_Queries.sql section J01`

3. Select section J01 and execute it. Expect one row: listing 1, SOLD, asking price CAD 283,250.00, two sellers and two offers. The query reads the payload JSONB column in tre_oo.listing_snapshot.

`OO_JSONB_Mapping/JSONB_Queries.sql section J02`

4. Select section J02 and execute it. Expect 13 ACTIVE CONDO listings. The first is listing 102 at CAD 456,500.00. Point to @> for nested containment and #>> for field extraction.

Narration The object model represents one Listing with a property, an agent, sellers and offers. These are composition relationships. The same object is serialized to JSON and stored in the snapshot table. This first JSONB query reads nested fields and counts the sellers and offers for listing one. It returns two sellers and two offers. The second query filters active condo listings and reads their asking prices, returning thirteen rows. JSONB makes a complete listing convenient to retrieve, but it duplicates summaries and can become stale. These snapshots need an explicit refresh after relational changes.

Closing

04:55-05:15 Return to the Data ownership table in README.md and finish the demonstration.

Narration The demonstration covers relational integrity and reporting, document-shaped notes, and object-to-JSONB mapping. The three representations have different roles, with clear data ownership. The package includes the scripts, explanations and recorded query results.

Optional transaction demonstration

`Query_Portfolio/Transaction_Examples.sql section T02`

Insert after Q12 and shift later timestamps by 30 seconds. Target total: 5 minutes 45 seconds. Select and run PRECHECK in Transaction_Examples.sql first: offer 127 should be SUBMITTED, its listing ACTIVE, and has_sale false.

Find T02 and execute two selections. First select its BEGIN through the first SELECT; show ACCEPTED, SOLD and 440000.00. Then select ROLLBACK and the following SELECT; show SUBMITTED, ACTIVE and has_sale false. Skip the final DO check. The opening file guard prevents running every example together. If interrupted, execute ROLLBACK.

Narration This optional example accepts an offer and completes its sale inside one transaction. The first result shows accepted and sold states. After rollback, the original submitted and active states return, and the temporary sale is gone. The demonstration leaves the business rows unchanged.

After recording, add the actual video URL to the Demo video section of README.md. Check that the recording lasts 4-6 minutes and that the SQL results, Firestore actions and JSONB results are readable.